ZAMBOANGA DEL SUR PROVINCIAL GOVERNMENT COLLEGE

Aurora Campus

Poblacion, Aurora, Zamboanga del Sur

College of Computing Studies

SMARTATTENDANCE: WEB-BASED STUDENT ATTENDANCE MANAGEMENT SYSTEM VIA DYNAMIC QR CODE SCANNING

Presented to the Faculty of the College of Computing Studies
In Partial Fulfillment of the Requirements in
Databases
Data Structure and Algorithm
Bachelor of Science in Information Systems

Submitted to:

SIR JUN GIE CASUYAC
Database Instructor

SIR APPLEJAN KILATON
Data Structure and Algorithm Instructor

Submitted by:

PAUL MARK EDULSORA
Main Author

CHRISTINE DURANGPARANG
KAREN JOY BUAGAS
ABDULRAKMAN DATU-ABDULLAH
JERYMEH TAÑOLA
Database Co-Authors

DONIE AMACNA
JV ASURO
Data Structure and Algorithm Co-Authors

October 2026

TABLE OF CONTENTS

1. Statement of the Problem **1**

2. General Objective **3**

3. Specific Objectives **3**

4. Scope **5**

4.1 User Accounts and Access Control 5

4.2 Superadministrator Module 6

4.3 Instructor Module 6

4.4 Student Module 7

4.5 Attendance Verification Pipeline 8

4.6 Data Management and Security 9

4.7 Development Tools and Technologies 9

4.8 Data Structures, Algorithms and Command-Line Interface 10

4.9 Testing, Deployment and Evaluation 10

5. Limitations **10**

5.1 Devices, Browsers and Connectivity 10

5.2 Attendance Verification 11

5.3 Functional Coverage 12

5.4 Hosting and Infrastructure 12

5.5 Data Privacy 13

5.6 Evaluation 13

6. Database Design **14**

6.1 Normalization 14

6.2 Entity-Relationship Diagram 14

6.3 Relationships 16

6.4 Data Dictionary 16

6.5 Integrity Rules 22

7. Conceptual Framework **24**

8. Data Structures and Algorithms **26**

8.1 Data Structures 27

8.2 Searching Algorithms 27

8.3 Sorting Algorithm 28
8.4 Counting 29
8.5 Effect on the Database Design 30
8.6 Command-Line Interface 30
9. System Flowcharts **31**
10. Testing **34**
10.1 Automated Tests 34
10.2 Functional Test Cases 35
10.3 Algorithm Performance 37
11. Screenshots **38**
12. Planned Additions and Major Changes **44**
12.1 Exit Record of Archived Students 45
12.2 Personal Contact Number 46
12.3 Clickable Dashboard Cards 46
12.4 Passkey Device Binding and Private-Mode Warning 47
12.5 Student Search and Report Page 48
12.6 Geofence Default of 250 Meters 48
12.7 QR Code Validity of 30 Seconds and Faster Roster Refresh 49
12.8 Pop-Up Notification for Unenrolled Scans 49
12.9 Password Recovery 50
12.10 Student Class Feed 50
12.11 Year Level Limited to 1 to 4 50
References **52**

1. STATEMENT OF THE PROBLEM

Attendance is one of the most basic yet consequential records that an academic institution keeps. It serves as evidence of student participation in instruction, forms part of class standing in many grading schemes, informs policies on retention and academic standing, and is among the earliest observable indicators that a student may be at risk of failing or discontinuing their studies. Because these decisions depend on attendance data, the data must be accurate, complete, timely and verifiable.

In many higher education institutions, attendance is still recorded through oral roll calls, paper sign-in sheets or class record notebooks kept individually by each instructor. These methods share several weaknesses. Roll calls consume instructional time that grows with class size and recurs in every meeting of every subject. Sign-in sheets can be signed on behalf of absent classmates, a practice commonly referred to as *proxy attendance,* and the instructor has no practical means of confirming that every signature belongs to a student who is physically present. Handwritten records must later be transcribed, which introduces errors, and because the records are dispersed among individual instructors, consolidating them for departmental or institutional monitoring is slow and laborious. Consequently, patterns of chronic tardiness and absenteeism are often recognized only after they have already affected a student's academic standing.

Quick Response (QR) code technology (ISO/IEC, 2015) has been adopted by some institutions to speed up the recording of attendance. However, a system that relies on a *static* QR code introduces a weakness of its own: the code can be photographed or captured in a screenshot and forwarded through messaging applications, allowing students who are not in the classroom to check in remotely. Speed alone, therefore, does not guarantee the integrity of attendance records. An effective solution must verify not only that a student scanned a valid code, but also that the scan was made during the class period, from the student's own registered device, from within the vicinity of the classroom, and by a student who is officially enrolled in the subject. It must also remain flexible enough to accommodate legitimate situations in which location cannot serve as evidence of presence, such as online or remote class sessions. Finally, such a system must protect institutional records from accidental or unauthorized deletion and must handle the personal information of students in a manner consistent with Republic Act No. 10173, otherwise known as the Data Privacy Act of 2012.

This study addresses these concerns through the design and development of **SmartAttendance**, a web-based student attendance management system for Zamboanga

del Sur Provincial Government College (ZDSPGC) that records classroom attendance through the scanning of dynamic, time-limited QR codes. Specifically, the study seeks to answer the following questions:

1. What problems do instructors, students and administrators of ZDSPGC encounter in the existing attendance monitoring process in terms of:
    - 1.1 the time consumed in recording attendance during class hours;
    - 1.2 the accuracy and integrity of attendance records;
    - 1.3 the occurrence and detection of proxy attendance;
    - 1.4 the storage, retrieval and consolidation of attendance records; and
    - 1.5 the timely identification of students with excessive tardiness and absences?
2. What features and security mechanisms must a web-based attendance system incorporate to ensure that each recorded attendance reflects the genuine presence of an enrolled student, particularly through:
    - 2.1 cryptographically signed, time-bound dynamic QR codes;
    - 2.2 the binding of each student account to a single device;
    - 2.3 location verification by means of geofencing that the instructor may enable or disable for each class session; and
    - 2.4 the validation of subject enrollment?
3. How can the system be designed to provide role-appropriate functions to the superadministrator, instructors and students while preserving institutional records through controlled account states and non-destructive data management?
4. How can attendance data be organized in a normalized relational database and transformed into meaningful punctuality information for instructors and administrators?
5. What is the level of acceptability of the developed system as evaluated by students, instructors and information technology (IT) experts in terms of the software product quality characteristics of ISO/IEC 25010, namely:
    - 5.1 functional suitability;
    - 5.2 performance efficiency;
    - 5.3 compatibility;
    - 5.4 usability;

5.5 reliability;

5.6 security;

5.7 maintainability; and

5.8 portability?

2. GENERAL OBJECTIVE

The general objective of this study is to design, develop and evaluate SmartAttendance, a secure web-based student attendance management system for Zamboanga del Sur Provincial Government College that records classroom attendance through the scanning of dynamic, time-limited QR codes; verifies the authenticity of every check-in through token validation, device binding, optional geofencing and enrollment validation; and provides role-based tools and punctuality analytics that enable administrators, instructors and students to monitor attendance accurately and efficiently.

3. SPECIFIC OBJECTIVES

Specifically, the study aims to:

1. Determine the problems encountered by instructors, students and administrators of ZDSPGC in the existing attendance monitoring process in terms of the time consumed in recording attendance, the accuracy and integrity of records, the occurrence and detection of proxy attendance, the storage, retrieval and consolidation of records, and the timely identification of students with excessive tardiness and absences.
2. Design and develop a web-based attendance management system with the following features:
    2.1 user registration and authentication with role-based access control for three user roles (superadministrator, instructor and student) and an account life cycle of *pending*, *active* and *suspended* states, in which instructor accounts require superadministrator approval and student accounts are activated automatically upon enrollment in a subject;
    2.2 a dynamic QR code module that encodes a JSON Web Token (JWT) (Jones et al., 2015) signed by the server, valid for ten (10) seconds and rotated every eight (8) seconds on the instructor's display, accompanied by a visible countdown;

2.3 a browser-based QR scanning module that uses the camera of the student's mobile device without requiring the installation of a native application;

2.4 an anti-proxy verification pipeline that executes, in strict sequence, token validation, session status validation, device binding that permits only one device per student and one student per device, geofence validation using the Haversine formula (Sinnott, 1984) with an instructor-defined radius, and enrollment validation, and that classifies each accepted check-in as *present* or *late*;

2.5 a per-session geofencing option that allows the instructor to enable location verification for face-to-face classes or to disable it for online and remote classes;

2.6 an audit trail that records every rejected scan together with the reason for rejection, the submitted device identifier, the reported coordinates and the raw request data;

2.7 session management tools for instructors, including a live attendance roster, manual status override, remarks, device unbinding, dropping a student from a subject, approval of scans by unenrolled students, automatic marking of absences when a session closes, export of session records to CSV, and export of the full class record of a subject to an Excel workbook;

2.8 punctuality analytics at the student, class and institution levels, including attendance rate, number of late arrivals, number of absences and a standing classification of *Good Standing*, *Frequent Tardy* or *Critical Absences*;

2.9 administration tools for the superadministrator, including the approval of instructor applications, the management of user accounts, subjects and departments, institution-wide monitoring of rejected scans, and institution-wide attendance reports exported to Excel; and

2.10 data structures and algorithms implemented in the program, namely a dictionary-based student index, a first-in, first-out queue of unenrolled scans, linear and binary search, a merge sort for sorting and ranking, and single-pass counting, together with a command-line interface that provides the basic record operations.

3. Design a relational database in the third normal form (Codd, 1972) that enforces referential integrity and preserves institutional records through non-destructive

(soft-delete) data management.

4. Implement security measures that protect the system and its personal data, including salted password hashing, protection against cross-site request forgery, rate limiting of scan requests, HTTPS-only session cookies and server-side enforcement of role-based permissions, in support of the requirements of the Data Privacy Act of 2012 (Republic Act No. 10173, 2012).
5. Test the functionality of the system through automated unit and integration tests that cover the verification pipeline, role-based access control and data management rules.
6. Deploy the system on a cloud-hosted web server that is accessible through a secure HTTPS connection.
7. Evaluate the level of acceptability of the system among students, instructors and IT experts using the software product quality characteristics of ISO/IEC 25010 (ISO/IEC, 2011).

4. SCOPE

This study covers the design, development, testing, deployment and evaluation of SmartAttendance, a web-based student attendance management system intended for Zamboanga del Sur Provincial Government College. The system covers the attendance monitoring activities of three groups of users: the **superadministrator**, who manages the system on behalf of the institution; the **instructors**, who conduct class sessions and manage the attendance of their classes; and the **students**, who record their own attendance by scanning QR codes. The system is accessed through a web browser on smartphones, tablets, laptops and desktop computers and uses a responsive interface designed primarily for mobile devices in the case of students and for laptops, tablets and projected displays in the case of instructors and the superadministrator.

4.1 User Accounts and Access Control

The system provides self-registration for instructors and students. Registration collects the user's name, username, e-mail address and password, and an address in the Philippine format of street address, barangay, city or municipality, province and postal code. Students additionally provide their student identification number, course or program, year level and emergency contact information. Users sign in with either their username or their e-mail address. Every new account starts in the *pending* state. A

pending instructor cannot use instructor functions until the superadministrator approves the application. A pending student may sign in and view the student dashboard, which displays a notice that the account is awaiting verification; the account becomes *active* automatically when an instructor enrolls the student in a subject or when the superadministrator approves the account. A *suspended* account is barred from signing in and from scanning. Superadministrator accounts are created by the system administrator through a command-line utility. Access to every page and application programming interface (API) endpoint is restricted on the server according to the user's role and account state.

4.2 Superadministrator Module

The superadministrator dashboard summarizes the number of students, active instructors, pending accounts, class sessions held on the current day, currently running sessions and rejected scans during the past seven days. It presents institution-wide present, late and absent rates, a watch list of students classified as *Frequent Tardy* or *Critical Absences*, and a breakdown of rejected scans by reason. The superadministrator can approve pending instructors and students; search, filter and edit user profiles; suspend, reactivate, archive and restore accounts; clear a student's device binding; create subjects and departments; reassign subjects to active instructors; activate or deactivate departments; and download an Excel attendance report, optionally limited to a date range and a department, with sheets for attendance by subject, the standing of every student and the rejected scans, as well as the Excel class record of any subject. The superadministrator can also review the complete audit log of rejected scans, filtered by reason, student or subject.

4.3 Instructor Module

The instructor dashboard lists the subjects assigned to the instructor together with class statistics. The instructor may enroll students in a subject by student identification number, username or e-mail address, either one at a time or by pasting a class list of up to two hundred identifiers, and may drop a student from a subject, which removes only the enrollment record and never the student's account. To start a class session, the instructor selects the subject, enters an optional session title and chooses a duration of fifteen (15) minutes to three (3) hours. The instructor also chooses after how many minutes a scan counts as late: ten (10), fifteen (15), which is the default, or thirty (30) minutes, or a custom value from one (1) to one hundred eighty (180) minutes that must be shorter than

the session duration. The instructor also decides whether geofencing applies to the session. When geofencing is enabled, the browser captures the instructor's current location and the instructor sets the allowed radius, which defaults to forty (40) meters and may range from ten (10) to one thousand (1,000) meters. When geofencing is disabled, as in online classes, no location is stored and student locations are not checked.

Once a session starts, the instructor's screen, which may be projected for the class, displays a QR code that is replaced every eight (8) seconds and a circular countdown to the next code. The same screen shows a live roster of checked-in students that refreshes every thirty (30) seconds or on demand; polling stops automatically when the session ends in order to conserve server resources. From the roster, the instructor can change a student's status to present, late or absent, add remarks such as a verified medical excuse, record attendance manually for a student who could not scan, unbind a student's device after confirmation, and drop a student from the subject. A separate feed lists, in the order of their first attempt (first come, first served), the students who scanned a valid code but are not enrolled in the subject, each with an *Add to Class and Approve* action that enrolls the student, activates a pending account and records the student's attendance in a single step. The instructor may open a punctuality profile of any student showing the attendance rate, number of late arrivals, number of absences and standing classification for that subject, and may export the attendance records of a session as a comma-separated values (CSV) file. The instructor may also download the class record of a subject as an Excel workbook with three sheets: a summary of each student's attendance, rate, standing and rank; an attendance grid that marks every student as present (P), late (L) or absent (A) in every session; and a list of the sessions with their counts. The class roster can be searched by name, identification number or username; sorted by name, identification number, attendance rate, number of late arrivals, number of absences or standing; and shows each student's rank by attendance rate. When a session is closed, either by the instructor or upon reaching its expiration time, every enrolled student without a record is marked absent automatically.

4.4 Student Module

The student module provides a scanner page that uses the camera of the student's device through the *html5-qrcode* library, with a scanning frame, a camera switch and a flashlight toggle on devices that support it. On first use, the browser generates a random device identifier and stores it permanently on the device; this identifier accompanies every scan request together with the student's current coordinates and the scanned token.

The student dashboard displays the student's total attendance, on-time percentage, number of late arrivals and number of absences, a per-subject summary and the complete attendance history, which may be filtered by subject.

4.5 Attendance Verification Pipeline

Every scan submitted by a student is evaluated on the server through the following checks, performed in strict sequence. A scan that fails any check is rejected with an explanatory message, and checks 1 to 5 additionally record the attempt in the audit log:

1. **Token validation.** The signature and expiration of the JWT are verified, allowing a tolerance of two (2) seconds for network delay. A tampered, malformed or expired token is rejected.
2. **Session validation.** The class session must still be open and must not have passed its expiration time.
3. **Device binding.** If the student's account has no bound device, the submitted device identifier is bound to it, provided that the device is not already bound to another student's account. If a different device is already bound to the student, or if the device belongs to another student, the scan is rejected as a device mismatch. Each device can therefore record attendance for only one student, and each student can use only one device.
4. **Geofence validation.** For sessions with geofencing enabled, the great-circle distance between the instructor's and the student's coordinates is computed with the Haversine formula, and a scan from beyond the allowed radius, or one without location data, is rejected. This check is skipped for sessions in which the instructor disabled geofencing.
5. **Enrollment validation.** The student must be enrolled in the subject of the session. Otherwise, no attendance record is created and the attempt appears in the instructor's feed of unenrolled scans.
6. **Recording.** The attendance is recorded as *present*, or as *late* if the scan occurs later than the number of minutes that the instructor set for the session (fifteen minutes by default). A repeated scan by a student who has already checked in is acknowledged without creating a duplicate record.

4.6 Data Management and Security

The study covers a relational database of nine application tables designed in the third normal form, which is presented in Section 6. Student and instructor accounts are never physically deleted; they are archived by setting their active flag to false so that attendance and audit histories are preserved, and physical deletion of users, subjects, sessions and logs is disabled in the administrative interface. Passwords are stored only as salted hashes. The system protects form and API submissions against cross-site request forgery, limits each student to thirty (30) scan requests per minute, issues session cookies only over HTTPS in production, and refuses to start in production without a secret key and an explicit list of allowed host names. The system stores no tuition, fee, billing or payment information, as the institution is fully government-funded.

4.7 Development Tools and Technologies

The system is developed with the technologies listed in Table 1.

Table 1

Development Tools and Technologies

Component	Technology
Programming language (server)	Python 3.12 or later
Web framework	Django 5.2 (Django Software Foundation, 2025)
Application programming interface	Django REST Framework
Token signing	PyJWT, HMAC-SHA256 (HS256) algorithm
QR code generation	segno (server-side SVG rendering)
QR code scanning	html5-qrcode (client-side, in the web browser)
User interface	HTML5, Tailwind CSS and vanilla JavaScript; Django templates
Location services	Browser Geolocation API; Haversine distance computed on the server
Database	SQLite for development and pilot deployment; PostgreSQL-compatible configuration
Data structures and algorithms	Hand-written merge sort, binary search, linear search, counting, hash index and FIFO queue (attendance/dsa.py)
Command-line interface	Django management commands attendance_cli and dsa_benchmark
Spreadsheet export	openpyxl (Excel .xlsx workbooks)
Hosting	PythonAnywhere (HTTPS subdomain)

4.8 Data Structures, Algorithms and Command-Line Interface

The study covers the implementation, within the program, of the data structures and algorithms used to search, sort, rank, count and queue attendance data: a list, a dictionary-based hash index, a first-in, first-out queue, linear search, binary search, merge sort and single-pass counting. These are described in Section 8. The study also covers a command-line interface that provides the basic record operations on the same database, and a benchmark command that measures the performance of the searching and sorting algorithms.

4.9 Testing, Deployment and Evaluation

The core logic of the system, including each stage of the verification pipeline, the role-based access rules, the account life cycle, the non-destructive data management rules, the data structures and algorithms and the command-line interface, is verified through a suite of eighty-five automated unit and integration tests (Section 10). For pilot implementation and demonstration, the system is deployed on a cloud platform as a service (PythonAnywhere) and is accessed through a secure HTTPS address. The acceptability of the system is evaluated by selected students, instructors and IT experts of ZDSPGC through a survey questionnaire based on the eight software product quality characteristics of ISO/IEC 25010 and rated on a five-point Likert scale.

5. LIMITATIONS

The following limitations define the boundaries within which the developed system and the findings of the study should be interpreted.

5.1 Devices, Browsers and Connectivity

1. Students must use a smartphone or tablet with a working camera and a modern web browser that supports camera access, the Geolocation API and local storage. Students without a compatible device cannot record their own attendance; their attendance must be entered manually by the instructor.
2. Both the instructor's display and the students' devices require an active internet connection throughout the session. The system has no offline mode, and scans cannot be stored on the device for later submission.
3. Browsers permit camera and location access only on pages served over HTTPS. The system therefore cannot be used for scanning over an unsecured connection

or on a local network address without a secure tunnel.

4. The style sheet and the QR scanning library are served by the system itself, but the Inter typeface is loaded from Google Fonts; where that service is blocked, devices display their own default fonts.
5. Some in-application browsers, such as those embedded in social media and messaging applications, may block camera access; students must open the system in a standard browser.

5.2 Attendance Verification

6. Geofencing depends on the location reported by the student's device. Indoor accuracy may degrade to several tens of meters, the system cannot determine the specific room or floor of a multi-storey building, and it cannot reliably detect locations falsified through mock-location applications. Geofencing therefore serves as a deterrent layer rather than conclusive proof of presence.
7. Device binding uses a random identifier generated and stored by the browser, because web browsers do not disclose hardware identifiers such as the International Mobile Equipment Identity (IMEI). Clearing browser data, using private browsing, switching to another browser or replacing the device produces a new identifier, which the system treats as a different device until an instructor or the superadministrator clears the binding. Section 12.4 describes a planned improvement.
8. The system verifies the device and the account but not the identity of the person holding the device; it does not employ biometric or facial recognition. Because each device can be bound to only one student account, a student cannot sign in to a classmate's account on their own phone and check in on the classmate's behalf; such an attempt is refused and recorded in the audit log. A proxy check-in would therefore require the absent student's own phone together with that student's credentials. The system cannot detect a student who physically hands both to a classmate, so instructor supervision and review of the audit log remain necessary.
9. The ten-second validity of each QR code reduces, but does not entirely eliminate, the possibility of relaying a code to another person in real time, particularly in sessions where geofencing is disabled. In such sessions, verification relies solely on the token, the device binding and the enrollment check.

5.3 Functional Coverage

10. Enrollment of students in subjects is performed within the system by instructors or the superadministrator, individually or by pasting a list of up to two hundred identifiers. The system is not integrated with the registrar's enrollment system, any student information system or any learning management system, and it cannot import class lists from files.
11. Attendance is classified only as present, late or absent. Lateness is measured from the moment the instructor starts the session, not from the scheduled start of the class, so an instructor who opens the session late also shifts the grace period. Excused absences are documented through remarks rather than as a separate status.
12. The system does not send SMS, e-mail or push notifications to students, parents, guardians or advisers.
13. Analytics are descriptive (rates, counts and standing classifications); the system does not provide predictive analytics. Exports are limited to CSV files per session and Excel workbooks per subject and for the institution; the system does not generate PDF reports or connect to external reporting tools.
14. The system does not cover tuition, fees, billing or payments; grades and class standing computation; class scheduling and room assignment; faculty attendance or payroll; and campus entry or gate monitoring.
15. The system is designed for a single institution and is not configured for multiple campuses or institutions with separate data. The interface is available in English only, and all times follow Philippine Standard Time.

5.4 Hosting and Infrastructure

16. The pilot deployment uses the free tier of PythonAnywhere, which provides a single web worker, 512 MiB of storage, a subdomain address and a web application that must be renewed monthly. The SQLite database used in the pilot is suited to a limited number of simultaneous users; institution-wide deployment would require paid hosting and a client-server database such as PostgreSQL.
17. Database backup is performed manually by downloading the database file; automated backup and disaster recovery are outside the scope of the study.

5.5 Data Privacy

18. The system processes personal data, including names, contact details, addresses, device identifiers and the location of each scan. The study implements technical safeguards but does not cover the organizational measures required by the Data Privacy Act of 2012, such as a privacy impact assessment, the designation of a data protection officer or registration with the National Privacy Commission, which remain the responsibility of the institution.

5.6 Evaluation

19. The evaluation is limited to the selected students, instructors and IT experts of ZDSPGC who participate during the pilot implementation period. The results reflect the perceptions of these respondents and may not be generalizable to other institutions or settings.

6. DATABASE DESIGN

The SmartAttendance database consists of nine application tables designed in the third normal form (3NF) (Codd, 1972). Figure 1 presents the entity-relationship diagram (ERD) of the database using crow's-foot notation, Table 2 summarizes the relationships between the entities and the referential action applied to each, and Tables 3 to 11 constitute the data dictionary. The diagram and the data dictionary were generated from the data models of the system and therefore correspond exactly to the tables of the implemented database.

6.1 Normalization

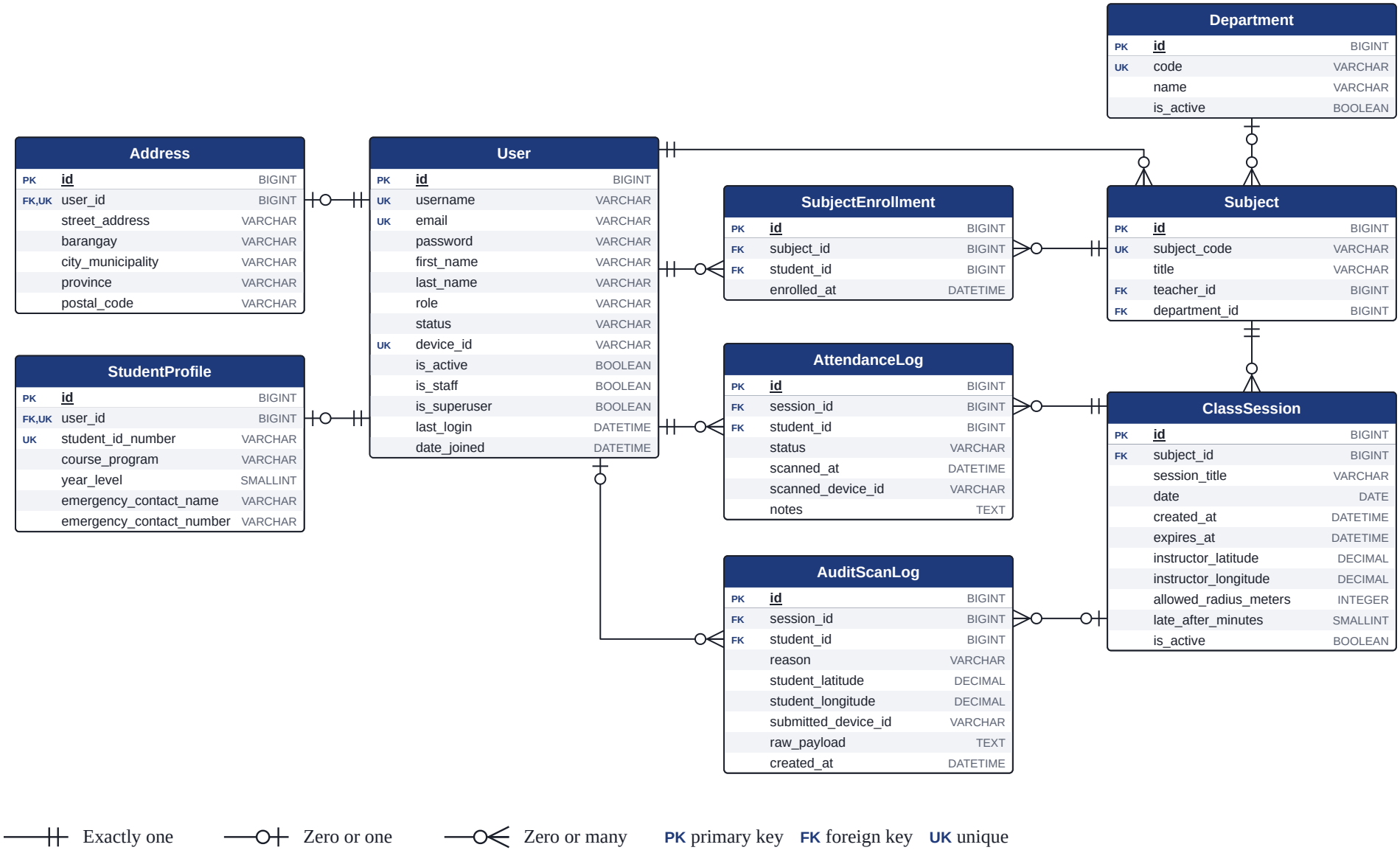
The database satisfies the first normal form because every attribute holds a single atomic value: the address, for instance, is decomposed into street address, barangay, city or municipality, province and postal code instead of being stored as a single text field, and enrollments and attendance are stored as individual rows rather than as lists. It satisfies the second normal form because every table uses a single-column surrogate primary key, while the natural keys of the two associative entities, `(subject_id, student_id)` in SubjectEnrollment and `(session_id, student_id)` in AttendanceLog, are enforced through composite unique constraints on which all other attributes depend in full. It satisfies the third normal form because no non-key attribute depends on another non-key attribute: the title of a subject is stored only in Subject and not repeated in sessions or logs, the program and year level of a student are stored only in StudentProfile, and derived values such as attendance rates, late counts and standing classifications are computed when requested rather than stored. Attributes that apply only to students are separated into StudentProfile in a one-to-one relationship with User, which avoids columns that would remain empty for instructors and administrators.

6.2 Entity-Relationship Diagram

Figure 1, on the following page, shows the nine entities with their attributes, keys and logical data types. Each relationship line connects a parent entity to the child entity that holds the foreign key. The symbol nearest an entity states how many instances of that entity may participate in the relationship: two bars denote exactly one, a circle and a bar denote zero or one, and a circle and a crow's foot denote zero or many.

Figure 1

Entity-Relationship Diagram of the SmartAttendance Database (Crow's-Foot Notation)



Note. Generated from the data models of the system (`attendance/models.py`). Logical data types are shown; SQLite maps them to its storage classes. Framework tables created by Django are omitted.

6.3 Relationships

Table 2 lists the eleven relationships of the database, the foreign key that implements each one, its cardinality and the referential action applied when a parent row is deleted.

Table 2

Relationships Among the Entities of the SmartAttendance Database

Parent	Child	Relationship	Foreign key	Cardinality	On delete
User	Address	has	`user_id`	1 : 0..1	CASCADE
User	StudentProfile	has	`user_id`	1 : 0..1	CASCADE
User (instructor)	Subject	teaches	`teacher_id`	1 : 0..N	CASCADE
Department	Subject	offers	`department_id`	0..1 : 0..N	SET NULL
Subject	SubjectEnrollment	has	`subject_id`	1 : 0..N	CASCADE
User (student)	SubjectEnrollment	enrolls in	`student_id`	1 : 0..N	CASCADE
Subject	ClassSession	holds	`subject_id`	1 : 0..N	CASCADE
ClassSession	AttendanceLog	records	`session_id`	1 : 0..N	CASCADE
User (student)	AttendanceLog	checks in	`student_id`	1 : 0..N	CASCADE
ClassSession	AuditScanLog	flags	`session_id`	0..1 : 0..N	CASCADE
User (student)	AuditScanLog	attempts	`student_id`	0..1 : 0..N	SET NULL

Note. “On delete” is the referential action the database applies to child rows if a parent row is physically deleted. Because the application archives records instead of deleting them, these actions serve as safeguards.

6.4 Data Dictionary

Tables 3 to 11 describe every column of the nine application tables. PK denotes a primary key, FK a foreign key and UK a unique key; the Null column indicates whether the database accepts a null value for the column.

Table 3

Data Dictionary of the User Entity (`attendance_user`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key, generated automatically.
`username`	`VARCHAR(150)`	UK	No	Unique sign-in name.
`email`	`VARCHAR(254)`	UK	No	Unique e-mail address; may also be used to sign in.
`password`	`VARCHAR(128)`		No	Salted PBKDF2-SHA256 hash of the password; the plain password is never stored.
`first_name`	`VARCHAR(150)`		No	Given name.
`last_name`	`VARCHAR(150)`		No	Surname.
`role`	`VARCHAR(20)`		No	User role: admin (superadministrator), instructor or student.
`status`	`VARCHAR(20)`		No	Account state: pending (default), active or suspended.
`device_id`	`VARCHAR(255)`	UK	Yes	Device identifier bound on the first successful scan; null when no device is bound. Unique among bound devices, so a device belongs to only one account.
`is_active`	`BOOLEAN`		No	False when the account is archived (soft delete).
`is_staff`	`BOOLEAN`		No	Grants access to the Django administration site.
`is_superuser`	`BOOLEAN`		No	Grants all framework-level permissions.
`last_login`	`DATETIME`		Yes	Date and time of the most recent sign-in.
`date_joined`	`DATETIME`		No	Date and time the account was created.

Note. Stores every account of the system: superadministrators, instructors and students.

Table 4

Data Dictionary of the Address Entity (`attendance_address`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`user_id`	`BIGINT`	FK, UK	No	Owner of the address (User); one address per user.
`street_address`	`VARCHAR(255)`		No	House number, street or purok; may be blank.
`barangay`	`VARCHAR(100)`		No	Barangay; may be blank.
`city_municipality`	`VARCHAR(100)`		No	City or municipality.
`province`	`VARCHAR(100)`		No	Province.
`postal_code`	`VARCHAR(10)`		No	Postal (ZIP) code; may be blank.

Note. Stores one address per user in the Philippine format.

Table 5

Data Dictionary of the StudentProfile Entity (`attendance_studentprofile`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`user_id`	`BIGINT`	FK, UK	No	Student account (User) that owns the profile.
`student_id_number`	`VARCHAR(50)`	UK	No	Institutional student identification number (e.g., 2024-0012).
`course_program`	`VARCHAR(150)`		No	Degree program (e.g., BS Information Systems).
`year_level`	`SMALLINT`		No	Current year level; defaults to 1.
`emergency_contact_name`	`VARCHAR(150)`		No	Name of the emergency contact person; may be blank.
`emergency_contact_number`	`VARCHAR(20)`		No	Telephone number of the emergency contact; may be blank.

Note. Stores the attributes that apply only to student accounts.

Table 6

Data Dictionary of the Department Entity (`attendance_department`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`code`	`VARCHAR(20)`	UK	No	Short department code (e.g., CCS).
`name`	`VARCHAR(150)`		No	Full name of the department.
`is_active`	`BOOLEAN`		No	False when the department is deactivated.

Note. Stores the academic departments that offer subjects.

Table 7

Data Dictionary of the Subject Entity (`attendance_subject`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`subject_code`	`VARCHAR(20)`	UK	No	Subject code (e.g., IS201).
`title`	`VARCHAR(200)`		No	Descriptive title of the subject.
`teacher_id`	`BIGINT`	FK	No	Instructor (User) assigned to the subject.
`department_id`	`BIGINT`	FK	Yes	Department that offers the subject; optional.

Note. Stores the subjects (classes) and their assigned instructors.

Table 8

Data Dictionary of the SubjectEnrollment Entity (`attendance_subjectenrollment`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`subject_id`	`BIGINT`	FK	No	Subject in which the student is enrolled; unique together with student_id.
`student_id`	`BIGINT`	FK	No	Enrolled student (User).
`enrolled_at`	`DATETIME`		No	Date and time of enrollment.

Note. Associative entity that records which students are enrolled in which subjects.

Table 9

Data Dictionary of the ClassSession Entity (`attendance_classsession`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`subject_id`	`BIGINT`	FK	No	Subject to which the session belongs.
`session_title`	`VARCHAR(150)`		No	Optional title (e.g., Week 7 – Use Case Diagrams).
`date`	`DATE`		No	Date of the class.
`created_at`	`DATETIME`		No	Start time of the session; lateness is measured from this time.
`expires_at`	`DATETIME`		No	Time after which scans are refused.
`instructor_latitude`	`DECIMAL(9,6)`		Yes	Latitude of the instructor when the session started; null when geofencing is disabled.
`instructor_longitude`	`DECIMAL(9,6)`		Yes	Longitude of the instructor when the session started; null when geofencing is disabled.
`allowed_radius_meters`	`INTEGER`		No	Geofence radius in meters; defaults to 40.
`late_after_minutes`	`SMALLINT`		No	Minutes after the start when a scan becomes late (10, 15, 30 or custom, 1–180); defaults to 15.
`is_active`	`BOOLEAN`		No	False once the session has been closed.

Note. Stores each attendance session together with its time window and geofence settings.

Table 10

Data Dictionary of the AttendanceLog Entity (`attendance_attendancelog`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`session_id`	`BIGINT`	FK	No	Class session; unique together with student_id.
`student_id`	`BIGINT`	FK	No	Student (User) whose attendance is recorded.
`status`	`VARCHAR(10)`		No	Attendance status: present (default), late or absent.
`scanned_at`	`DATETIME`		No	Date and time the record was created.
`scanned_device_id`	`VARCHAR(255)`		No	Device identifier used for the scan; blank for manual and automatic entries.
`notes`	`TEXT`		No	Remarks entered by the instructor (e.g., medical excuse verified).

Note. Associative entity that stores one attendance record per student per session.

Table 11

Data Dictionary of the AuditScanLog Entity (`attendance_auditscanlog`*)*

Column	Data type	Key	Null	Description
`id`	`BIGINT`	PK	No	Surrogate primary key.
`session_id`	`BIGINT`	FK	Yes	Session targeted by the scan, when it can be identified.
`student_id`	`BIGINT`	FK	Yes	Student (User) who attempted the scan.
`reason`	`VARCHAR(30)`		No	Reason for rejection: unenrolled_student, device_mismatch, out_of_bounds, expired_token or session_closed.
`student_latitude`	`DECIMAL(9,6)`		Yes	Latitude reported by the student's device.
`student_longitude`	`DECIMAL(9,6)`		Yes	Longitude reported by the student's device.
`submitted_device_id`	`VARCHAR(255)`		Yes	Device identifier submitted with the scan.
`raw_payload`	`TEXT`		No	Submitted request data, kept for review.
`created_at`	`DATETIME`		No	Date and time of the attempt.

Note. Stores every rejected scan for anti-proxy review.

6.5 Integrity Rules

In addition to primary and foreign keys, the database and the application enforce the following rules:

1. **One enrollment per student per subject.** A composite unique constraint on `(subject_id, student_id)` in SubjectEnrollment prevents duplicate enrollments.
2. **One attendance record per student per session.** A composite unique constraint on `(session_id, student_id)` in AttendanceLog prevents duplicate check-ins, including those caused by simultaneous submissions.
3. **One device per student and one student per device.** `User.device_id` is set on the first successful scan and may be cleared only by an instructor or the superadministrator; a null value means that no device is bound. A partial unique constraint on `device_id`, which applies only to bound devices, prevents two

accounts from being bound to the same device, and the verification pipeline refuses such a binding before it reaches the database.

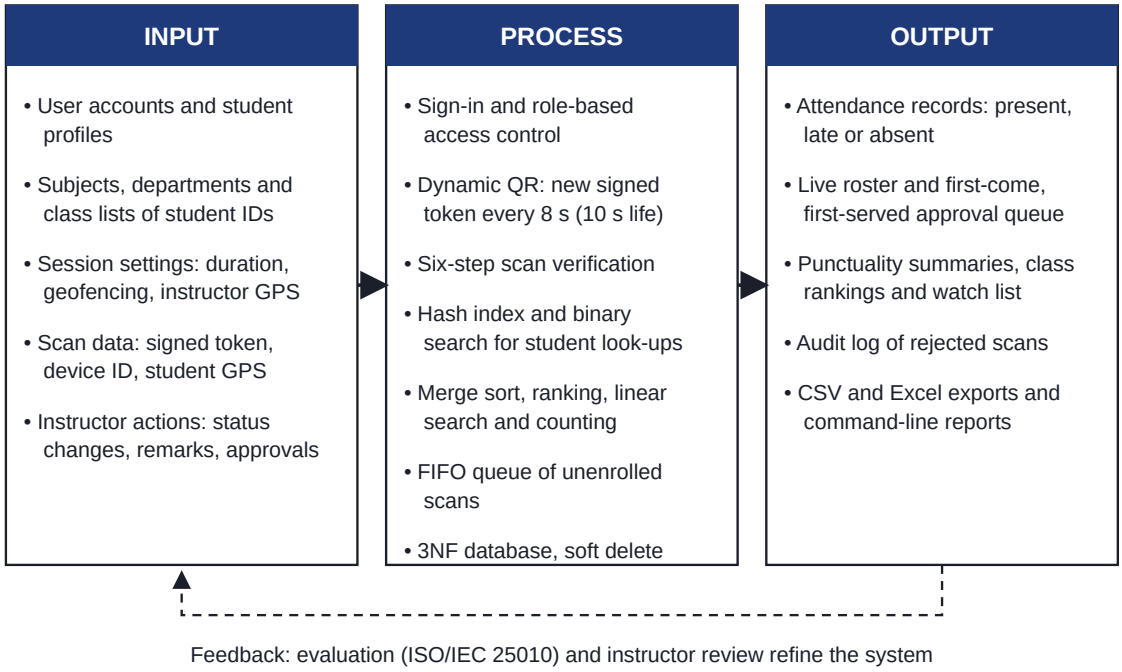
4. **Per-session geofencing.** Null values in `instructor_latitude` and `instructor_longitude` indicate that geofencing was disabled for that session.
5. **Preservation of audit history.** Because the student reference in AuditScanLog uses *SET NULL*, rejected-scan records would survive even if a user row were removed. In practice, the application never removes users, subjects, sessions or logs; accounts are archived through `is_active`, and the *CASCADE* rules serve only as a safeguard against orphaned rows.
6. **Excluded framework tables.** The tables that Django creates for its own operation (`auth_group`, `auth_permission`, `django_session`, `django_admin_log`, `django_content_type`, `django_migrations` and the link tables between users, groups and permissions) are present in the database but are not part of the application design and are therefore omitted from Figure 1.

7. CONCEPTUAL FRAMEWORK

The study adopts the Input–Process–Output (IPO) model as its conceptual framework. Figure 2 presents the inputs that SmartAttendance receives, the processes that transform them, and the outputs that the system produces for its users, together with the feedback that guides the refinement of the system.

Figure 2

Conceptual Framework of SmartAttendance (Input–Process–Output Model)



The **input** consists of the data supplied by the three groups of users: the accounts and profiles created during registration; the subjects, departments and class lists entered by the superadministrator and the instructors; the settings of each class session, namely its duration, whether geofencing applies and, if so, the instructor's location; the data submitted with every scan, which comprise the signed token read from the QR code, the device identifier and the student's location; and the actions of instructors, such as changes of attendance status, remarks and the approval of unenrolled students.

The **process** includes the authentication of users and the enforcement of role-based access; the generation of a new signed token every eight seconds for the dynamic QR code; the six-step verification of every scan; and the data structures and algorithms described in Section 8, which locate students through a hash index and binary search, sort

and rank records through merge sort, filter records through linear search, summarize attendance through single-pass counting, and order unenrolled scans in a first-in, first-out queue. All records are stored in a relational database in the third normal form and are archived rather than deleted.

The **output** comprises the attendance records classified as present, late or absent; the live roster and approval queue of each session; punctuality summaries, class rankings and the watch list of students who are not in good standing; the audit log of rejected scans; and the CSV and Excel exports and the reports produced through the web interface and the command-line interface. The evaluation of the system under ISO/IEC 25010 and the observations of instructors serve as **feedback** for its refinement.

8. DATA STRUCTURES AND ALGORITHMS

Although SmartAttendance stores its records in a relational database, the searching, sorting, ranking and counting operations listed in Table 12 are performed by algorithms that the researchers implemented in Python in the module `attendance/dsa.py`, rather than by the sorting and aggregation features of the database or by Python's built-in `sorted()` function. For these operations, the database only stores and retrieves the records. Each algorithm works on Python lists and dictionaries that are built from the retrieved records while a request is being processed, and its results are discarded afterwards. Table 12 summarizes the data structures and algorithms, where each one is used, why it is appropriate and its time complexity, where *n* denotes the number of records processed.

Table 12

Data Structures and Algorithms Implemented in SmartAttendance

Structure / algorithm	Where it is used	Why it is appropriate	Time complexity
List (dynamic array)	Every set of records loaded from the database: students, roster rows, attendance logs	Keeps records in order and gives direct access by position, which binary search and merge sort require	Access O(1); append O(1) amortized
Dictionary (hash index)	Look-up of students by username or e-mail; subjects by code; tallies per student and per session	Finds an entry by its key without scanning the whole collection	Look-up and insertion O(1) on average
FIFO queue	Feed of students who scanned but are not enrolled	Serves students in the order in which they arrived (first come, first served)	Enqueue and dequeue O(1)
Linear search	Roster search box; command-line search by part of a name, username or e-mail	Works on unsorted data and supports partial (substring) matches	O(n)
Binary search	Finding a student by identification number during single and bulk enrollment and in the command-line search	Locates an exact key in a sorted list by halving the search range at every step	O(log n)
Merge sort	Roster sorting options, class ranking, watch list, live roster, command-line lists	Guaranteed O(n log n) in every case and stable, so records with equal keys keep their order	O(n log n); O(n) extra space
Counting (single pass)	Present, late and absent totals, attendance rates and standing classifications	Tallies all categories in one pass over the records using a dictionary	O(n)

Structure / algorithm	Where it is used	Why it is appropriate	Time complexity
Competition ranking	Rank column of the class roster and command-line rankings	Assigns equal ranks to equal attendance rates (1, 2, 2, 4) after sorting	O(n) after sorting

8.1 Data Structures

List. Python's list is a dynamic array that stores references to records in consecutive positions. Every collection that the system processes, such as the students of a class or the attendance logs of a subject, is first loaded into a list. Because an element can be read by its position in constant time, the list is the natural input for binary search, which repeatedly inspects the middle position, and for merge sort, which divides the list by position.

Dictionary (hash index). A dictionary stores key–value pairs in a hash table, so that a value can be found from its key in constant time on average. The system builds a `HashIndex` of students keyed by username and e-mail address, so that an instructor can enroll a student by either value without a scan of all students. Dictionaries also hold the running tallies used for counting, for example `{"present": 15, "late": 2, "absent": 5}` for one student, and map each student to his or her attendance record when a session roster is displayed.

Queue. A queue is a first-in, first-out (FIFO) structure: elements are added at the back and removed from the front. When students who are not enrolled in a subject scan its code, each student is placed in a queue in the order of his or her first attempt, and a set records who is already queued so that repeated scans do not create duplicate entries. The instructor's screen numbers the queue and marks the first student as next, and the command-line menu approves students strictly from the front of the queue. The queue is implemented with `collections.deque`, which removes the first element in constant time, whereas removing the first element of a list requires shifting every remaining element.

8.2 Searching Algorithms

Linear search examines the records one by one and returns every record in which the search text appears in any of the selected fields, ignoring letter case. It is used where the data are not sorted by the searched value and where partial matches are required; for

example, typing *ramir* in the roster search box returns the student named Ramirez. Its running time grows in direct proportion to the number of records, O(n).

```
LINEAR-SEARCH(items, text, fields)
    matches ← empty list
    for each item in items
        for each field in fields
            if text appears in field(item) (ignoring case)
                append item to matches
                stop checking the other fields of this item
    return matches
```

Binary search finds an exact identification number in a list that has been sorted by identification number. It compares the target with the middle element and discards the half that cannot contain the target, so that the remaining range is halved at each step. For n students it needs at most $\lfloor \log_2 n \rfloor + 1$ comparisons: ten comparisons for a thousand students and twenty for a million. The list is sorted once with merge sort when the student directory is built; afterwards every look-up, including each of the up to two hundred identifiers of a pasted class list, uses binary search. Bulk enrollment of k students among n therefore costs O($n \log n + k \log n$), compared with O($k \times n$) for repeated linear searches.

```
BINARY-SEARCH(A, T)                       // A is sorted by key
    low ← 0;  high ← length(A) − 1
    while low ≤ high
        mid ← ⌊(low + high) / 2⌋
        if key(A[mid]) = T then return mid
        if key(A[mid]) < T then low ← mid + 1    // keep the right half
        else high ← mid − 1                     // keep the left half
    return −1                                    // not found
```

8.3 Sorting Algorithm

Merge sort is a divide-and-conquer algorithm (Cormen et al., 2022). It divides the list into two halves, sorts each half recursively, and merges the two sorted halves by repeatedly taking the smaller of their front elements. The list is halved about $\log_2 n$ times and each level of merging makes at most n comparisons, so the algorithm runs in O($n \log n$) time in the best, average and worst cases. Merge sort was chosen over simpler algorithms such as bubble sort and insertion sort, whose worst case is O(n^2), because the number of students to be ranked across the institution can grow large. It was also chosen because it is *stable*: when two keys are equal, the element from the left half is taken first, so records with equal keys keep their original order.

```
MERGE-SORT(L)
    if length(L) ≤ 1 then return L
    middle ← ⌊length(L) / 2⌋
    left ← MERGE-SORT(L[0 .. middle − 1])
    right ← MERGE-SORT(L[middle .. end])
    merged ← empty list;  i ← 0;  j ← 0
    while i < length(left) and j < length(right)
        if key(left[i]) ≤ key(right[j])         // "≤" keeps the sort stable
            append left[i] to merged;  i ← i + 1
        else
            append right[j] to merged;  j ← j + 1
    append the remaining elements of left and right to merged
    return merged
```

Each sorting option uses a composite key, which merge sort compares field by field. Sorting the roster by attendance rate, for instance, uses the key (−attendance rate, surname, given name), which places the highest rate first and orders students with equal rates alphabetically. The watch list uses (standing priority, attendance rate, name), so that students with critical absences appear first, followed by those with frequent tardiness, each group ordered from the lowest attendance rate. After the roster is sorted by attendance rate, competition ranking assigns rank 1 to the highest rate and gives students with equal rates the same rank.

8.4 Counting

The present, late and absent totals that appear on every dashboard are computed by a single pass over the attendance records. For each record, the dictionary entry for its status is incremented. To produce the figures of every student in one pass, the system counts pairs of (student, status) into a dictionary of dictionaries, after which the tally of any student is obtained in constant time. For a class with m students and n attendance records, this requires $O(n + m)$ time instead of a separate count for every student. The attendance rate, the on-time rate and the standing classification are then calculated from these tallies.

```
COUNT-BY-GROUP(pairs)                    // e.g. (student, "late")
    groups ← empty dictionary
    for each (group, value) in pairs
        if group is not in groups
            groups[group] ← {present: 0, late: 0, absent: 0}
        groups[group][value] ← groups[group][value] + 1
    return groups
```

8.5 Effect on the Database Design

The data structures and algorithms do not alter the database. No table or column was added for them, and Django's migration check confirms that the schema is unchanged. The hash index, the sorted list of identification numbers and the queue are rebuilt from existing tables whenever they are needed; tallies, rates, rankings and standings are derived values that are computed on request and never stored. Storing such derived values would introduce redundancy and transitive dependencies, so computing them in the program preserves the third normal form described in Section 6.1.

8.6 Command-Line Interface

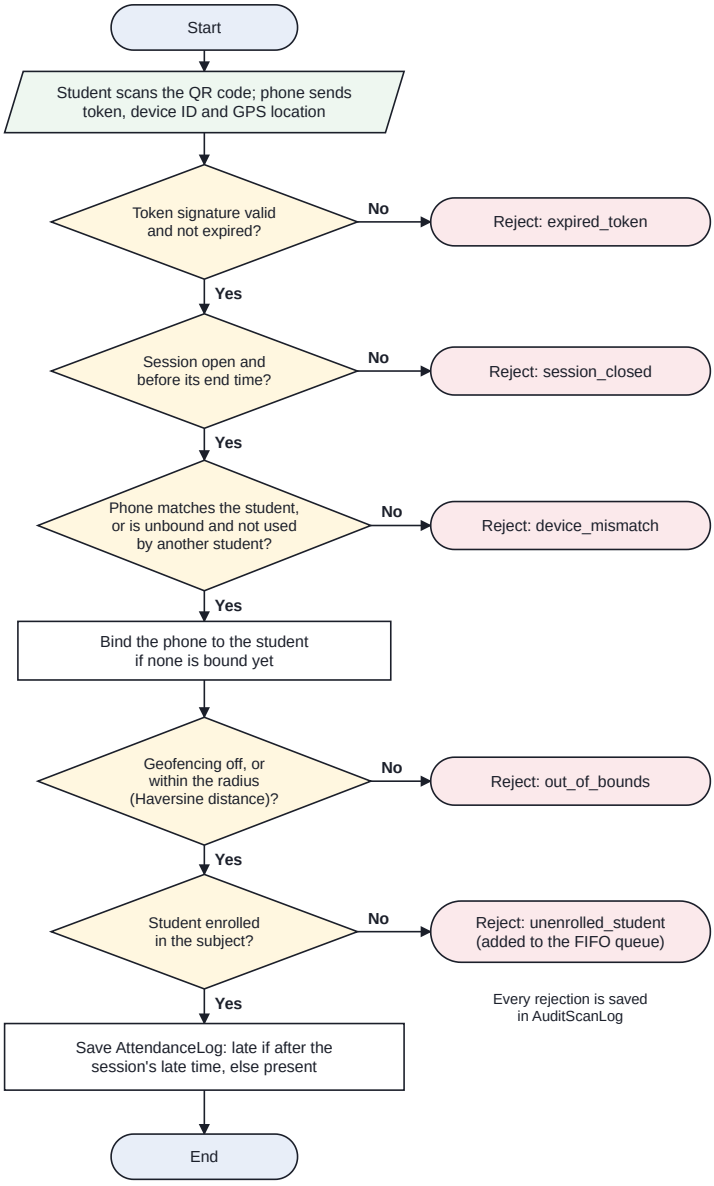
In addition to the web interface, the system provides a command-line interface, started with `python manage.py attendance_cli`, that operates on the same database and business rules. Its menu supports adding, viewing, searching, updating and archiving students; enrolling students by identification number (singly or as a list) and dropping them from subjects; listing sessions with their attendance counts and recording or updating attendance; producing school-wide and per-subject summaries, class rankings and the watch list; viewing and approving the queue of unenrolled scans; and an algorithm demonstration that compares linear and binary search and reports the number of comparisons made by merge sort. Every input is validated, invalid entries are explained and asked again, and errors during an operation are reported without terminating the program. A second command, `python manage.py dsa_benchmark`, measures the algorithms on generated data (Section 10.3).

9. SYSTEM FLOWCHARTS

Figures 3 to 5 present the flowcharts of the attendance verification process and of the two algorithms on which the search and ranking features depend. Rounded rectangles denote the start and end of a process, parallelograms denote input, rectangles denote processing steps and diamonds denote decisions.

Figure 3

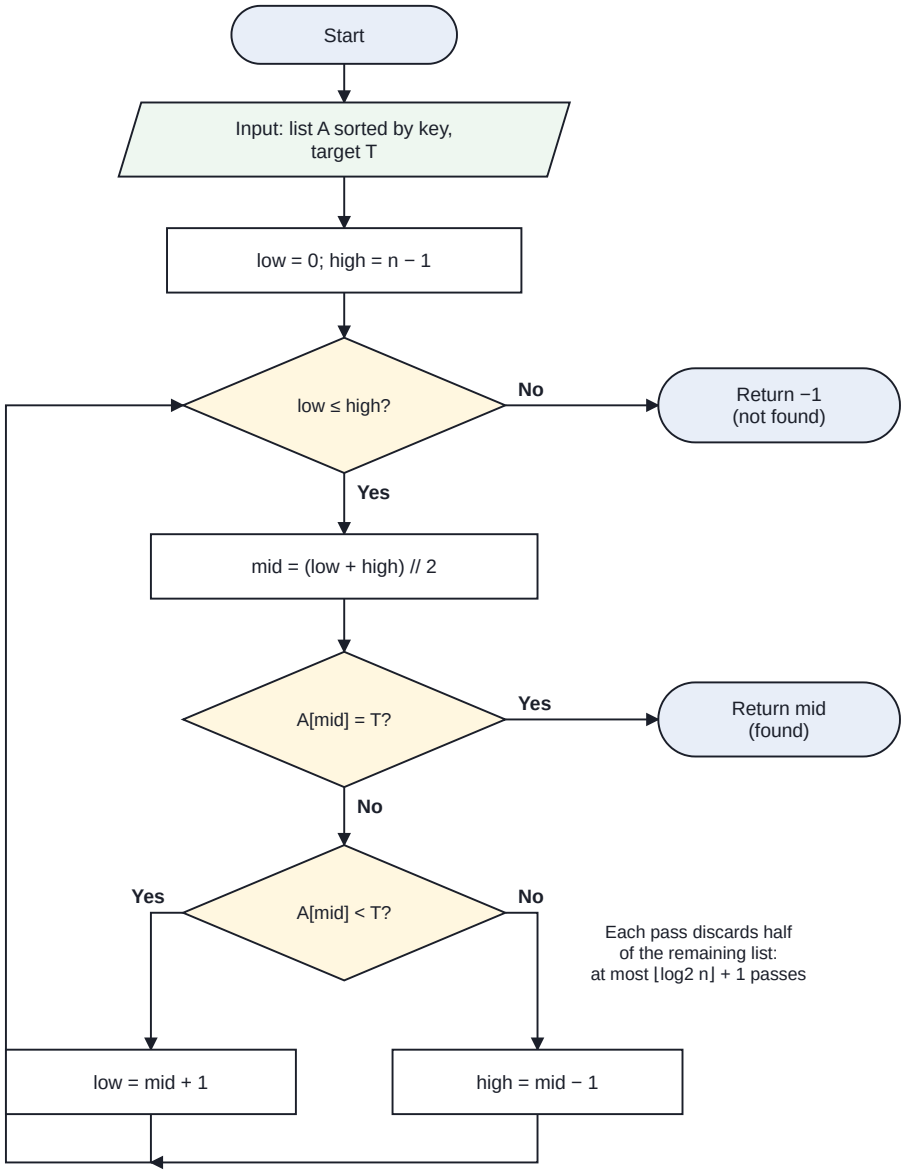
Flowchart of the Attendance Scan Verification Process



Note. Implemented in `verify_and_record_scan` (`attendance/services.py`).

Figure 4

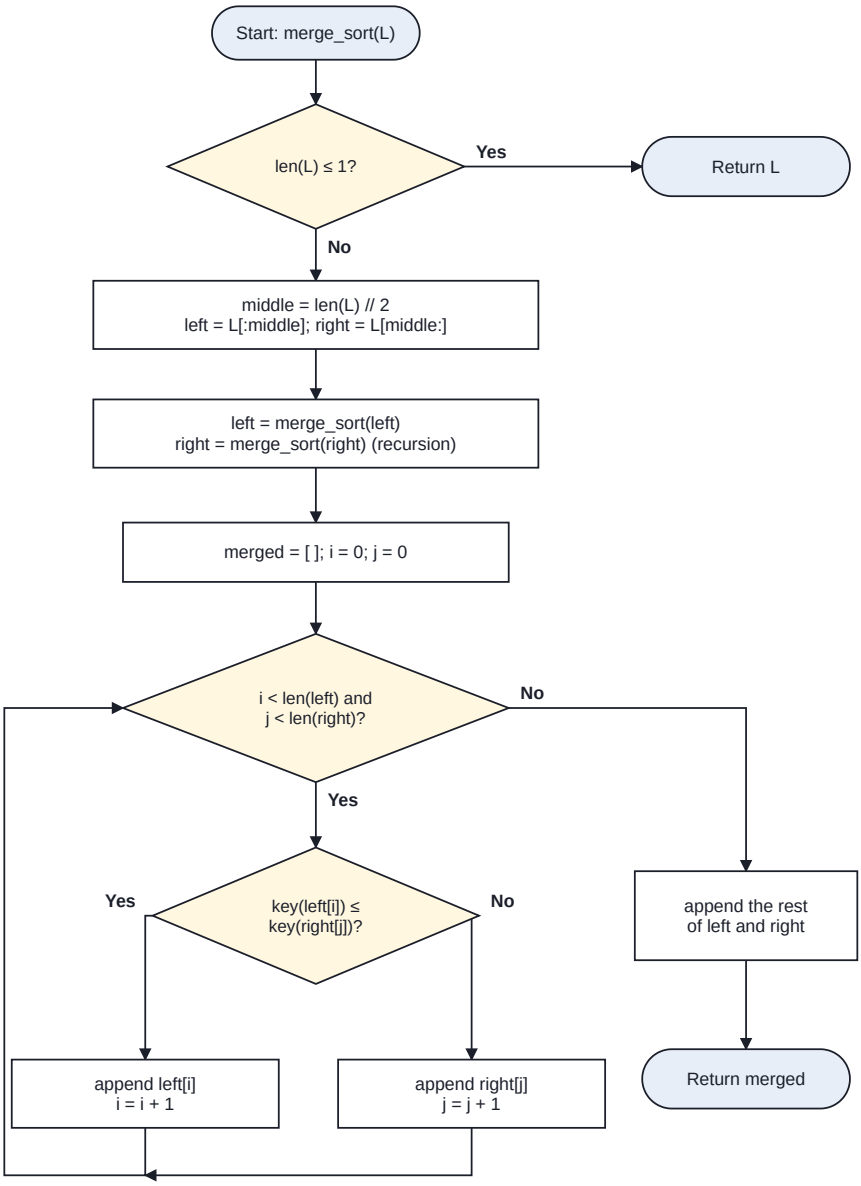
Flowchart of Binary Search



Note. Implemented in `binary_search` (`attendance/dsa.py`).

Figure 5

Flowchart of Merge Sort



Note. Implemented in `merge_sort` (`attendance/dsa.py`).

10. TESTING

10.1 Automated Tests

The system was tested through 85 automated unit and integration tests written with Django's testing framework and executed with `python manage.py test`. Each test creates its own temporary database, so the tests do not affect real records. Table 13 groups the tests by the part of the system they verify. All 85 tests passed.

Table 13

Automated Tests by Area

Area	What the tests verify	Tests	Result
Scan verification pipeline	Token, session, device binding (including one phone per student), geofence and enrollment checks; late threshold; duplicate scans; Haversine distance	17	All passed
Instructor actions and sessions	Starting sessions with or without geofencing, late time per session (10, 15, 30 or custom), QR token endpoint, status and remarks, unbinding, dropping, approving unenrolled scans, automatic absences	16	All passed
Accounts and access control	Registration states, approval, suspension, archiving (soft delete), role-based page access	8	All passed
Pages	Every public, superadministrator, instructor and student page renders for the correct role; seed data	9	All passed
Algorithms and data structures	Merge sort (correctness, stability, multi-field keys, n log n bound), linear and binary search, counting, ranking, queue, hash index, benchmark	14	All passed
Features that use the algorithms	Counting matches the database, watch list order, roster sort, search and rank, bulk enrollment, directory look-ups, FIFO queue of unenrolled scans	10	All passed
Excel exports	Instructor class record and Superadmin report: sheets, ranking order, totals matching the database, P/L/A grid, filters, date validation, access rules, download buttons	6	All passed
Command-line interface	Full add-search-update-enroll-record-report-archive workflow, invalid input, rejected registrations, end of input, algorithm demonstration	5	All passed
Total		**85**	**All passed**

10.2 Functional Test Cases

Table 14 lists representative functional test cases. Each case corresponds to one or more of the automated tests in Table 13, and the actual result was obtained by executing those tests.

Table 14

Representative Functional Test Cases

ID	Test case	Expected result	Actual result
TC-01	Enrolled student scans a current code inside the radius	Attendance recorded as present; phone bound	As expected (pass)
TC-02	Student scans 16 minutes after the session started	Recorded as late	As expected (pass)
TC-03	Token signed with a wrong key, or expired	Rejected; logged as expired_token	As expected (pass)
TC-04	Student scans with a second phone	Rejected with a device mismatch; no record	As expected (pass)
TC-05	Classmate signs in on a phone bound to another student	Rejected (phone already in use); no record	As expected (pass)
TC-06	Scan about 500 m away with geofencing on	Rejected as out of bounds	As expected (pass)
TC-07	Scan from any location with geofencing off	Recorded; device and enrollment checks still applied	As expected (pass)
TC-08	Unenrolled student scans	No record; student added to the queue in arrival order	As expected (pass)
TC-09	Class list with valid and unknown identifiers	Valid students enrolled; unknown identifiers reported	As expected (pass)
TC-10	Roster sorted by attendance rate	Rates in descending order; equal rates share a rank	As expected (pass)
TC-11	Roster search for “ramir”	Only the matching student is listed	As expected (pass)
TC-12	Instructor drops a student from a subject	Enrollment removed; account and history kept	As expected (pass)
TC-13	Command-line menu given 9, “abc” and an empty entry	Error message shown; menu asked again; no crash	As expected (pass)
TC-14	Command-line registration with a used username and mismatched passwords	Student not added; reasons listed	As expected (pass)
TC-15	Session started with “late after” set to 10 minutes; student scans at 11 minutes	Recorded as late	As expected (pass)
TC-16	Custom late time of 30 minutes on a 30-minute session	Refused: must be shorter than the duration	As expected (pass)
TC-17	Instructor downloads the Excel class record	Workbook with Summary, Attendance grid and Sessions sheets; totals match the database	As expected (pass)

ID	Test case	Expected result	Actual result
TC-18	Superadmin report with an end date before the start date	No file; message asks for a valid range	As expected (pass)

10.3 Algorithm Performance

The command `python manage.py dsa_benchmark` generated lists of 1,000, 10,000 and 100,000 unique student identification numbers in random order, sorted each list once with merge sort, and searched for 201 identifiers (200 that exist and one that does not) with linear search on the unsorted list and with binary search on the sorted list. Table 15 presents the results. The numbers of comparisons are exact; the times depend on the computer and are shown only for comparison. Merge sort stayed below the bound of $n \log_2 n$ comparisons. Linear search needed about half of n comparisons on average, while binary search needed at most $\lfloor \log_2 n \rfloor + 1$, so that at 100,000 students binary search used roughly 3,000 times fewer comparisons.

Table 15

Comparisons and Running Time of the Searching and Sorting Algorithms

Students (n)	Merge sort comparisons	$n \log_2 n$	Merge sort (ms)	Linear search avg. comparisons	Binary search avg. comparisons	Linear search (ms)	Binary search (ms)
1,000	8,734	9,966	1.88	554.7	9.0	5.34	0.24
10,000	120,513	132,877	24.39	5,113.7	12.4	52.73	0.36
100,000	1,536,360	1,660,964	343.8	50,034.0	15.7	763.42	0.49

Note. Times are the best of five runs (totals for 201 look-ups), measured in the development environment; they vary between computers, while the numbers of comparisons do not.

11. SCREENSHOTS

Figures 6 to 14 present screenshots of the system running with the demonstration data provided by the `seed_data` command.

Figure 6

Sign-In Page with the School Logo and Campus Photograph

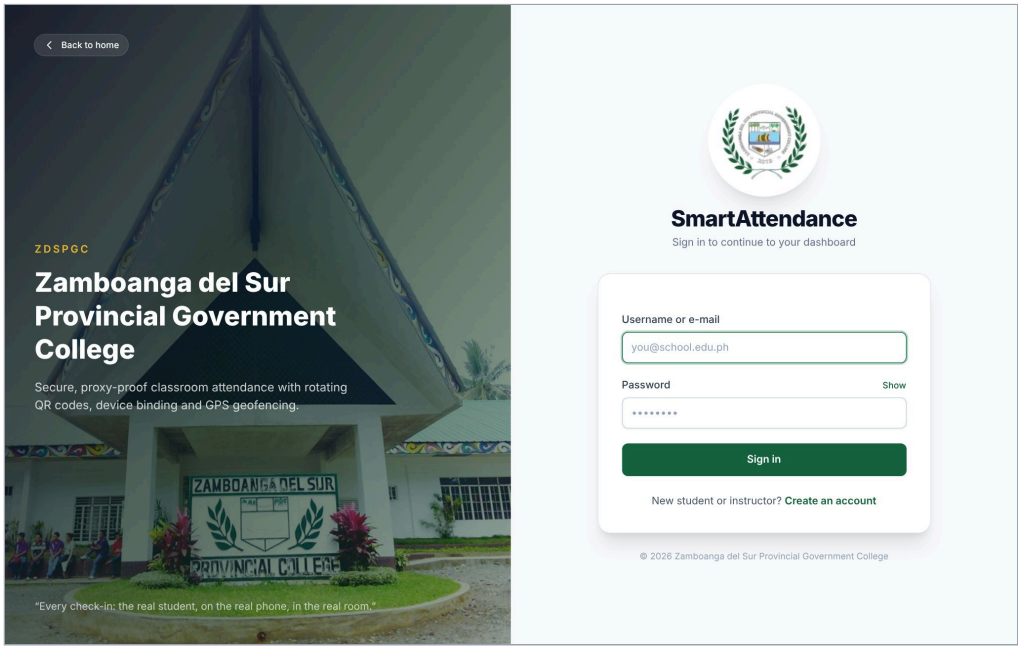


Figure 7

Instructor Dashboard with Assigned Subjects and Class Statistics

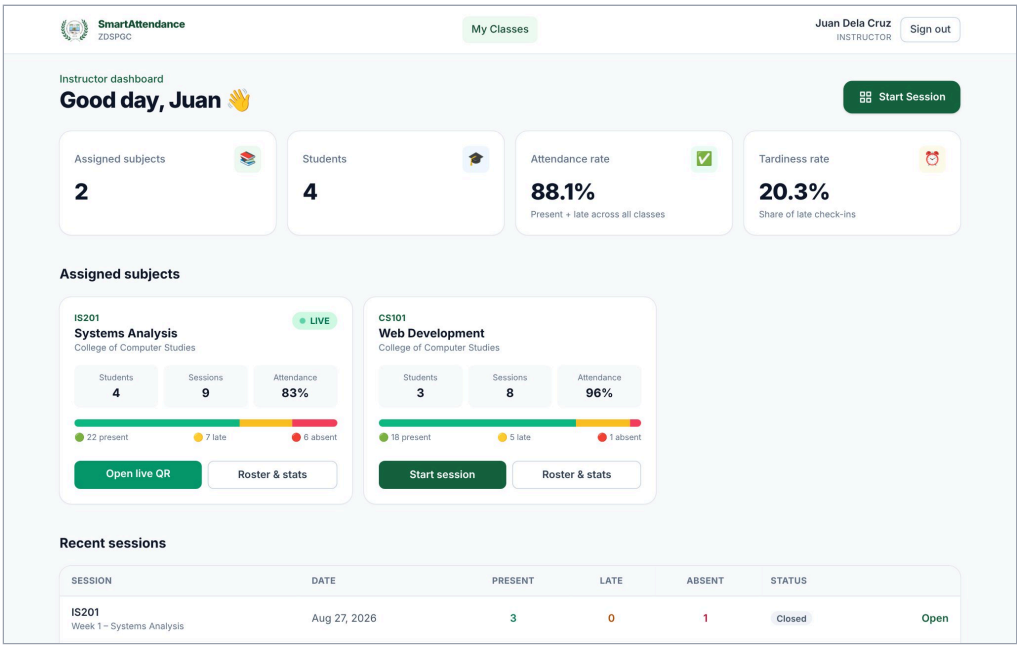


Figure 8

Live Session Screen with the Rotating QR Code, Countdown, Counts, Approval Queue and Security Alerts

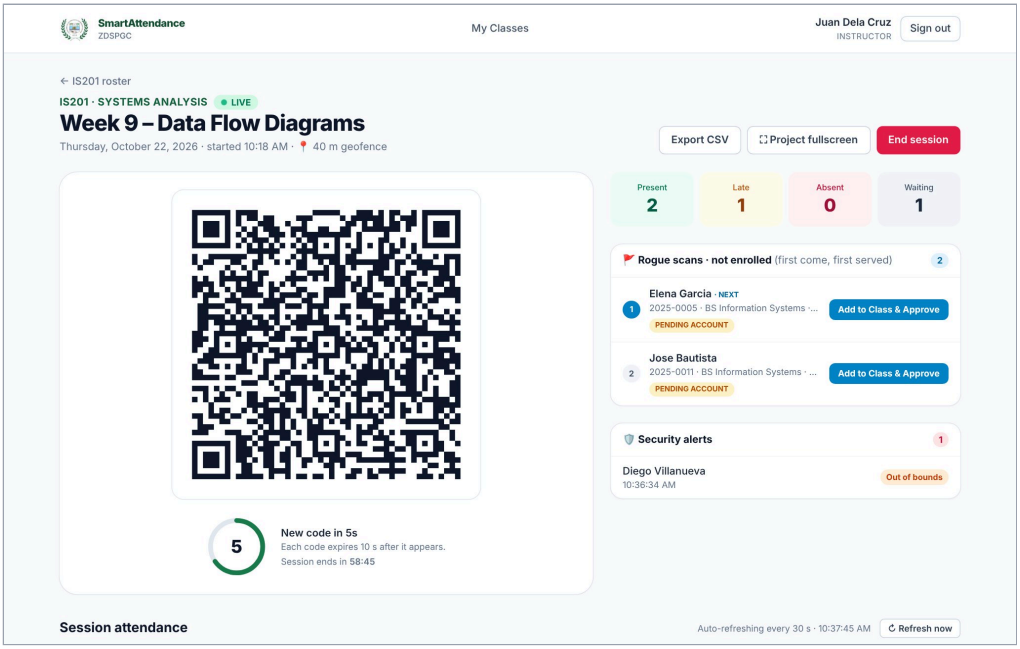


Figure 9

Class Roster Sorted by Attendance Rate with Rank, Search and Bulk Enrollment

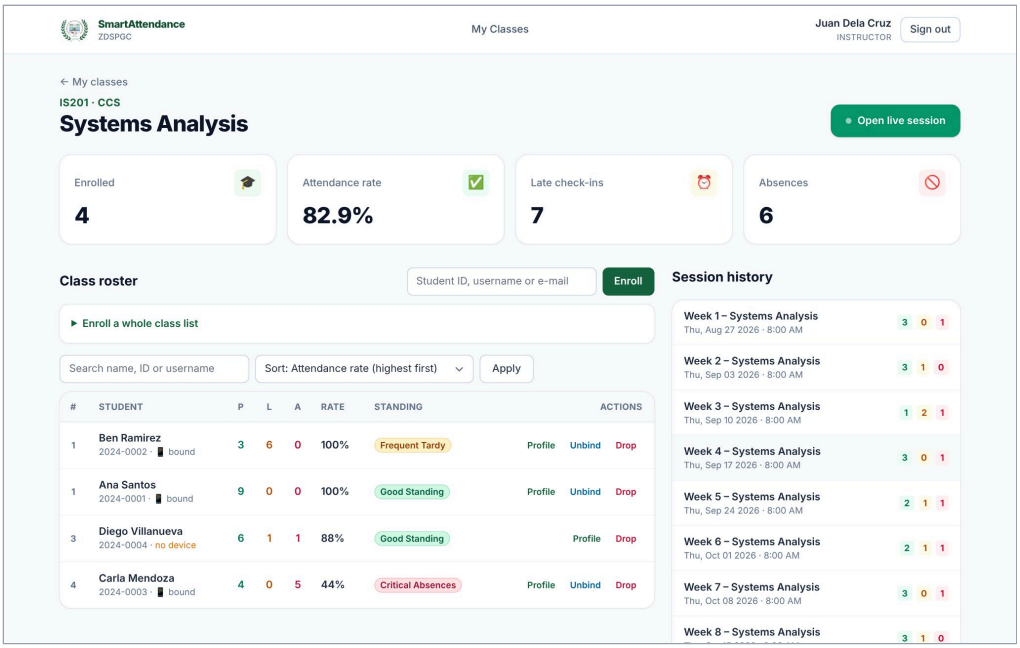


Figure 10

Superadministrator Dashboard with Institution-Wide Punctuality and the Watch List

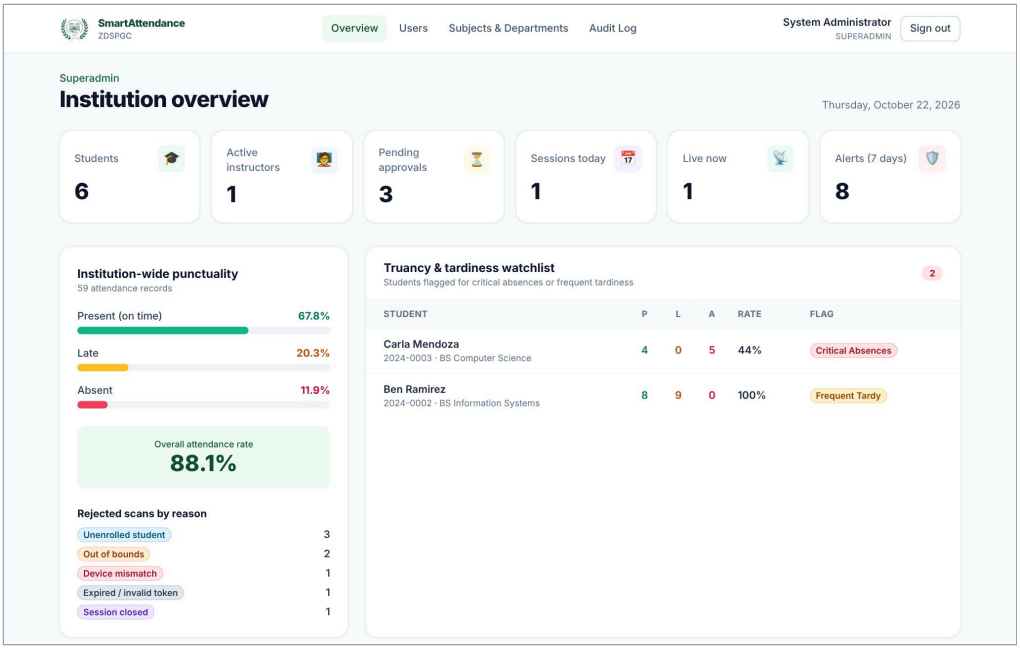


Figure 11

Student Dashboard and QR Scanner on a Mobile Phone

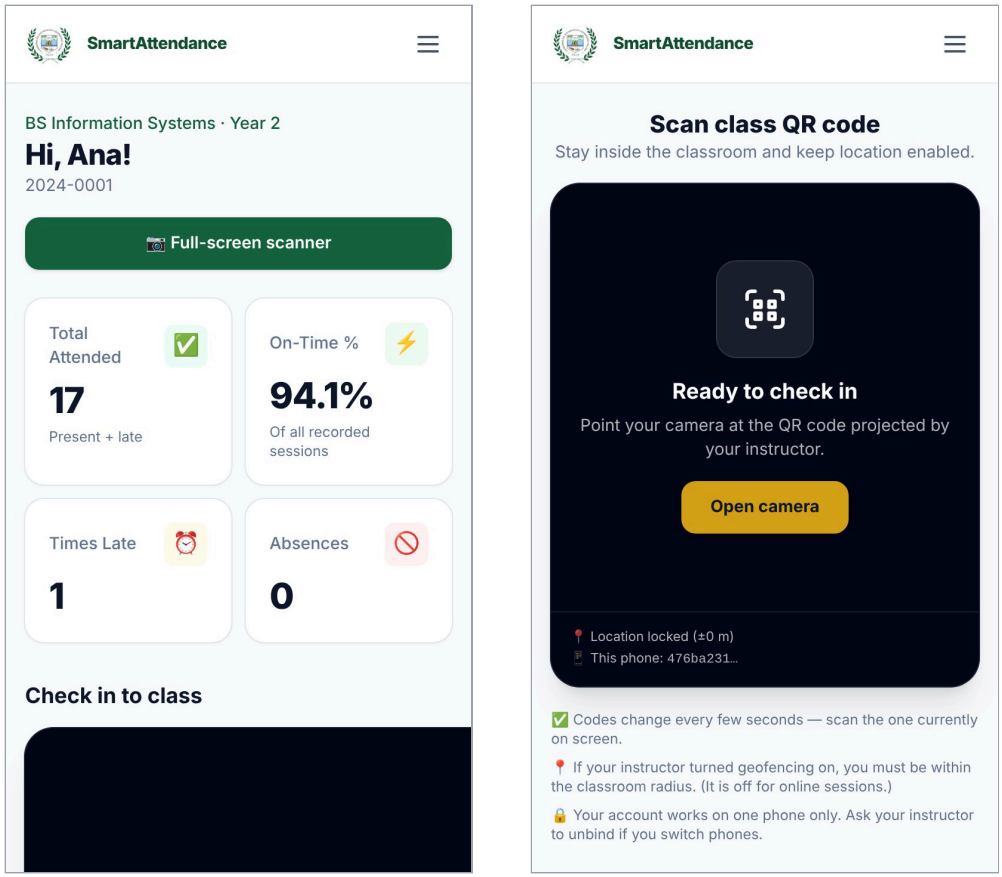
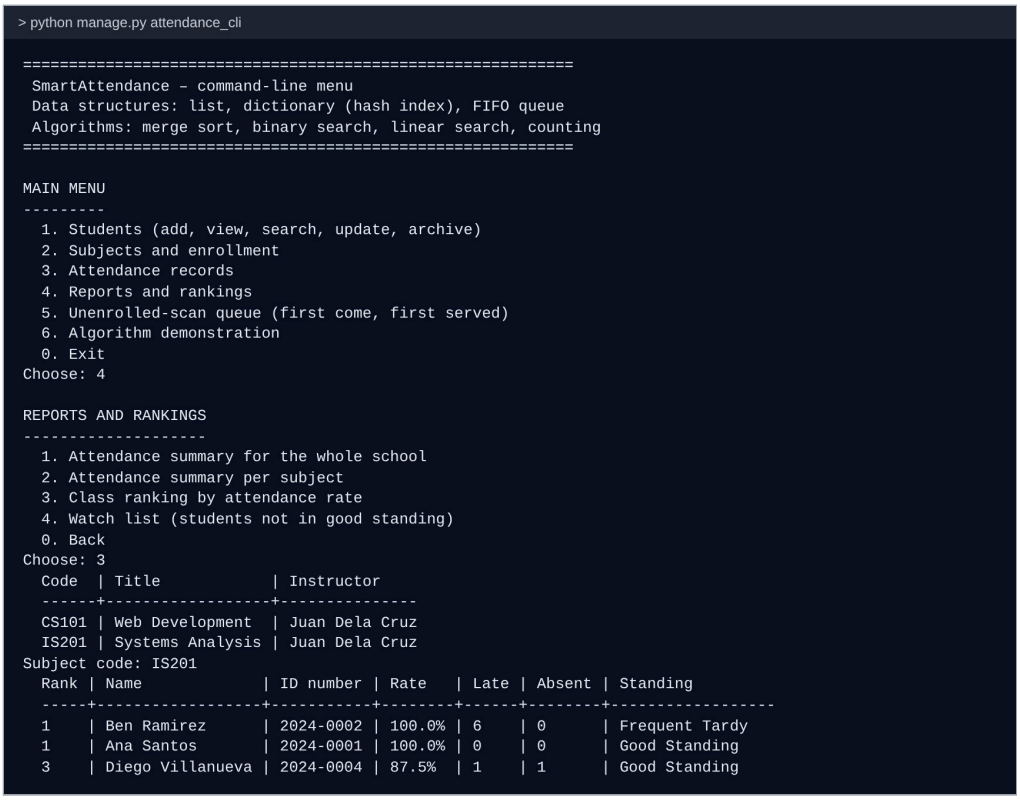


Figure 12

Command-Line Interface: Class Ranking by Attendance Rate (Merge Sort)



```
> python manage.py attendance_cli

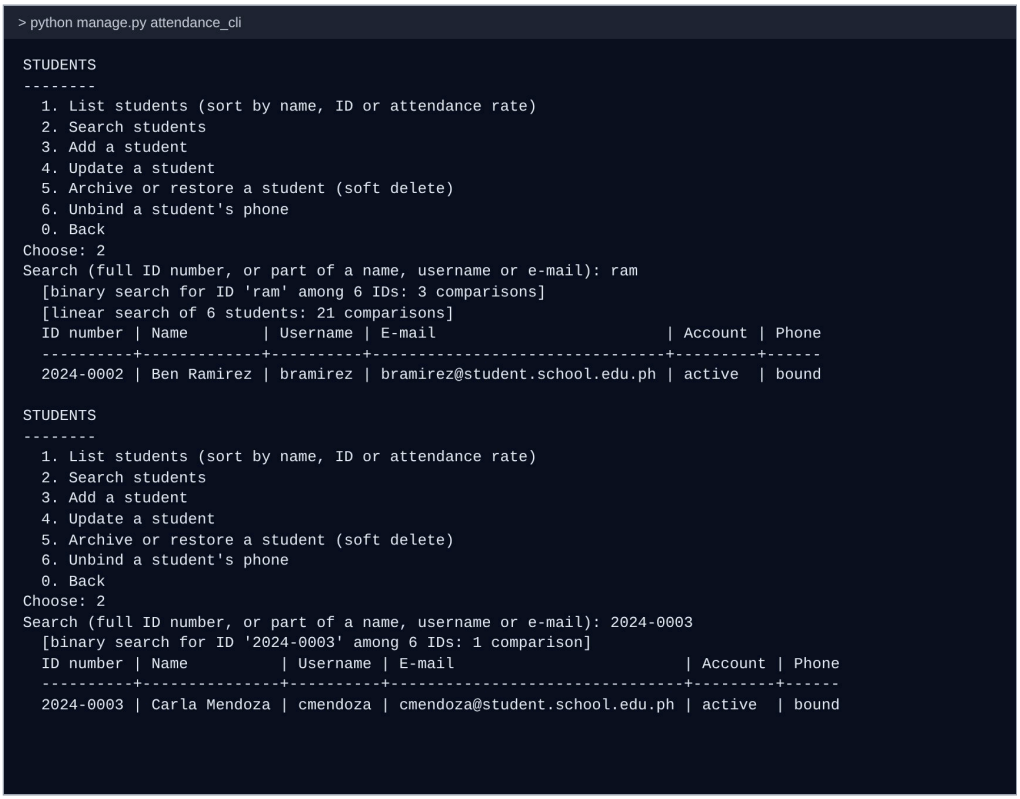
============================================================
 SmartAttendance – command-line menu
 Data structures: list, dictionary (hash index), FIFO queue
 Algorithms: merge sort, binary search, linear search, counting
============================================================

MAIN MENU
---------
  1. Students (add, view, search, update, archive)
  2. Subjects and enrollment
  3. Attendance records
  4. Reports and rankings
  5. Unenrolled-scan queue (first come, first served)
  6. Algorithm demonstration
  0. Exit
Choose: 4

REPORTS AND RANKINGS
--------------------
  1. Attendance summary for the whole school
  2. Attendance summary per subject
  3. Class ranking by attendance rate
  4. Watch list (students not in good standing)
  0. Back
Choose: 3
  Code  | Title            | Instructor
  ------+------------------+--------------
  CS101 | Web Development  | Juan Dela Cruz
  IS201 | Systems Analysis | Juan Dela Cruz
Subject code: IS201
  Rank | Name             | ID number | Rate   | Late | Absent | Standing
  -----+------------------+-----------+--------+------+--------+-----------------
  1    | Ben Ramirez      | 2024-0002 | 100.0% | 6    | 0      | Frequent Tardy
  1    | Ana Santos       | 2024-0001 | 100.0% | 0    | 0      | Good Standing
  3    | Diego Villanueva | 2024-0004 | 87.5%  | 1    | 1      | Good Standing
```

Figure 13

Command-Line Interface: Linear Search by Partial Name and Binary Search by Identification Number



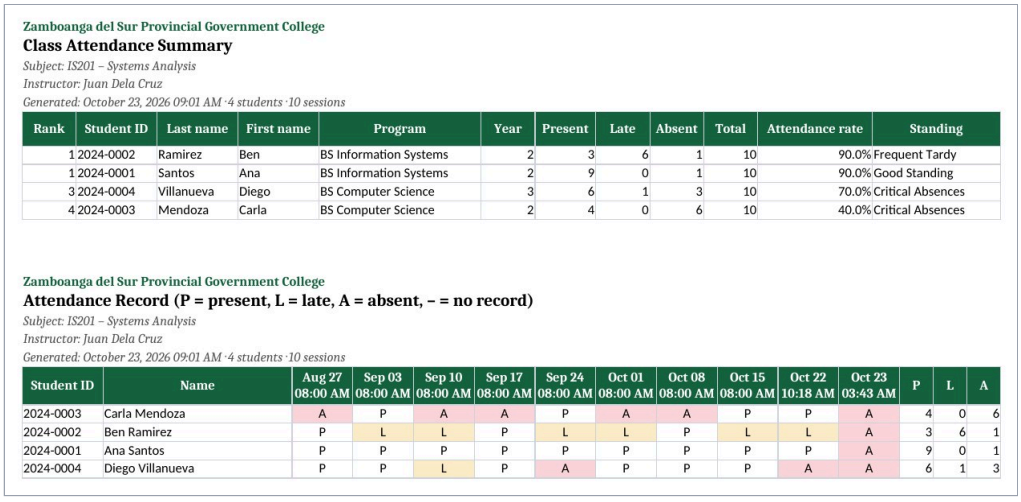
```
> python manage.py attendance_cli

STUDENTS
--------
  1. List students (sort by name, ID or attendance rate)
  2. Search students
  3. Add a student
  4. Update a student
  5. Archive or restore a student (soft delete)
  6. Unbind a student's phone
  0. Back
Choose: 2
Search (full ID number, or part of a name, username or e-mail): ram
  [binary search for ID 'ram' among 6 IDs: 3 comparisons]
  [linear search of 6 students: 21 comparisons]
  ID number | Name        | Username | E-mail                         | Account | Phone
  ----------+-------------+----------+--------------------------------+---------+------
  2024-0002 | Ben Ramirez | bramirez | bramirez@student.school.edu.ph | active  | bound

STUDENTS
--------
  1. List students (sort by name, ID or attendance rate)
  2. Search students
  3. Add a student
  4. Update a student
  5. Archive or restore a student (soft delete)
  6. Unbind a student's phone
  0. Back
Choose: 2
Search (full ID number, or part of a name, username or e-mail): 2024-0003
  [binary search for ID '2024-0003' among 6 IDs: 1 comparison]
  ID number | Name          | Username | E-mail                          | Account | Phone
  ----------+---------------+----------+---------------------------------+---------+------
  2024-0003 | Carla Mendoza | cmendoza | cmendoza@student.school.edu.ph | active  | bound
```

Figure 14

Excel Class Record Downloaded by an Instructor (Summary and Attendance Grid Sheets)



Zamboanga del Sur Provincial Government College

Class Attendance Summary

Subject: IS201 – Systems Analysis
Instructor: Juan Dela Cruz
Generated: October 23, 2026 09:01 AM · 4 students · 10 sessions

Rank	Student ID	Last name	First name	Program	Year	Present	Late	Absent	Total	Attendance rate	Standing
1	2024-0002	Ramirez	Ben	BS Information Systems	2	3	6	1	10	90.0%	Frequent Tardy
1	2024-0001	Santos	Ana	BS Information Systems	2	9	0	1	10	90.0%	Good Standing
3	2024-0004	Villanueva	Diego	BS Computer Science	3	6	1	3	10	70.0%	Critical Absences
4	2024-0003	Mendoza	Carla	BS Computer Science	2	4	0	6	10	40.0%	Critical Absences

Zamboanga del Sur Provincial Government College

Attendance Record (P = present, L = late, A = absent, – = no record)

Subject: IS201 – Systems Analysis
Instructor: Juan Dela Cruz
Generated: October 23, 2026 09:01 AM · 4 students · 10 sessions

Student ID	Name	Aug 27 08:00 AM	Sep 03 08:00 AM	Sep 10 08:00 AM	Sep 17 08:00 AM	Sep 24 08:00 AM	Oct 01 08:00 AM	Oct 08 08:00 AM	Oct 15 08:00 AM	Oct 22 10:18 AM	Oct 23 03:43 AM	P	L	A
2024-0003	Carla Mendoza	A	P	A	A	P	A	A	P	P	A	4	0	6
2024-0002	Ben Ramirez	P	L	L	P	L	L	P	L	L	A	3	6	1
2024-0001	Ana Santos	P	P	P	P	P	P	P	P	P	A	9	0	1
2024-0004	Diego Villanueva	P	P	L	P	A	P	P	P	A	A	6	1	3

12. PLANNED ADDITIONS AND MAJOR CHANGES

The review of the developed system and its live demonstration identified eleven additions and changes that are planned for a future version of SmartAttendance. They are not part of the version described in Sections 4 to 11 and are presented here so that the direction of further development is documented. Changes 6, 7 and 10 respond directly to the live demonstration, in which the geofence and the ten-second QR code validity delayed or prevented the check-in of students who were present; they deliberately favor a fast and efficient attendance session for instructors over the strictest anti-proxy settings. Table 16 summarizes the planned changes and their effect on the database design, and Sections 12.1 to 12.11 describe each of them. Every planned change keeps the database in Third Normal Form: new facts are stored either as a column that depends only on the key of its table or in a new table of their own, and no existing table or relationship is altered.

Table 16

Planned Additions and Major Changes and Their Effect on the Database

No.	Planned addition	Purpose	Effect on the database
1	Exit record of archived students	Records why a student left the institution (graduated, transferred, dropped or other), with the date and remarks.	One new table, exit_record, with foreign keys to user.
2	Personal contact number	Stores one contact number for each student and instructor, separate from the student's emergency contact.	One new column, contact_number, in user. No new table.
3	Clickable dashboard cards	Lets users open the detailed page behind each summary card on the dashboards.	None.
4	Passkey device binding, browser identifier as backup, and private-mode warning	Makes device binding survive the clearing of browser data and requires the phone to be unlocked for each scan.	One new table, passkey_credential, with a foreign key to user. The device_id column is kept for the backup method.
5	Student search and report page	Gives the superadministrator read-only, filtered student reports that use the implemented searching, sorting and counting algorithms.	None.
6	Geofence default of 250 meters with allowance for location accuracy	Prevents students who are present from being refused because of inaccurate location readings.	None. The radius is already stored per session.

No.	Planned addition	Purpose	Effect on the database
7	QR code valid for 30 seconds and instructor roster refreshed every 10 seconds	Reduces expired-code refusals on slow connections and shows check-ins to the instructor sooner.	None.
8	Pop-up notification for unenrolled scans with Approve and Refuse	Alerts the instructor to an unenrolled student at once and allows the scan to be accepted or refused.	One new column marking a refused scan in audit_scan_log.
9	Password recovery by e-mail, with a superadministrator reset link as last resort	Lets students and instructors who forget their password regain access without the superadministrator's help.	None. Uses the existing e-mail, password and account status columns.
10	Student class feed	Shows students which classmates have checked in and their own status during a session.	None.
11	Year level limited to 1 to 4	Matches the four-year programs offered by the institution.	None. Validation rule only.

12.1 Exit Record of Archived Students

In the current version, a student who graduates or transfers to another school is archived by the superadministrator. Archiving is a soft delete: it sets `is_active` to false, which prevents the student from signing in, scanning or being enrolled, while all enrollments, attendance logs and audit logs are preserved. The system does not, however, record why the student left, so a graduate and a transferee both appear only as archived.

The planned version adds an `exit_record` table with the columns `exit_record_id` (primary key), `student_id` (foreign key to `user`), `reason` (graduated, transferred, dropped or other), `exit_date`, `remarks`, `recorded_by_id` (foreign key to the superadministrator in `user`) and `created_at`. When the superadministrator archives a student, the system asks for the reason, the date and optional remarks and saves them together with the archiving. The reason is shown on the student's account page, the institution Excel report gains an exit records sheet, and the archive option of the command-line interface also asks for the reason. A student may have more than one exit record, for example a transferee who returns and later graduates; restoring an account keeps its earlier exit records as history.

Every column of the new table depends only on its primary key, and no detail of the student is copied into it, so the table satisfies Third Normal Form. The existing tables are not changed.

12.2 Personal Contact Number

The current version stores an emergency contact name and number for each student, which belong to another person such as a parent or guardian. Students and instructors have no contact number of their own; for instructors, the e-mail address is the only contact detail.

The planned version adds an optional `contact_number` column to the `user` table, so that students, instructors and the superadministrator each have exactly one contact number. Because each person has only one number, the number depends only on the user's key and is stored as a single column; a separate table would be needed only if a person could have several numbers. The column is stored as text of up to twenty characters rather than as a number, so that the leading zero and the country code are kept. Only Philippine mobile numbers in the forms 09XXXXXXXXX and +639XXXXXXXXX are accepted, and every number is saved in the single form +639XXXXXXXXX so that the same number cannot be stored in two ways. The column is not unique, because members of the same family may share a phone. Consistent with the verification policy of the system, the number may be entered at sign-up but can be changed afterward only by the superadministrator. The emergency contact columns of `student_profile` remain unchanged.

12.3 Clickable Dashboard Cards

The dashboards present summary cards such as the number of pending approvals and security alerts on the superadministrator dashboard, the assigned subjects and students on the instructor dashboard, and the times late and absences on the student dashboard. In the current version these cards display values only. The planned version makes each card a link to the page that lists the records behind it, already filtered: for example, the pending approvals card opens the user list filtered to pending accounts, the security alerts card opens the audit log, and the times late card opens the student's attendance history filtered to late records. The cards to be linked will be finalized after a review of the student, instructor and superadministrator dashboards. The change affects only the interface; it uses the existing pages and filters, leaves the access rules unchanged, so that instructors still see only their own subjects, and requires no change to the database.

12.4 Passkey Device Binding and Private-Mode Warning

Limitation 7 explains that device binding currently relies on a random identifier kept in the browser's storage, because browsers do not disclose hardware identifiers. Clearing the browser data or using private browsing produces a new identifier, which the system refuses as a different device until an instructor clears the binding. Hardware identifiers such as the IMEI and the Wi-Fi MAC address cannot be used instead: browsers do not expose them, the MAC address does not reach the server, and current Android and iOS devices present a randomized MAC address to each network. The planned version therefore strengthens device binding in three parts.

1. **Passkey required to scan.** On a phone that supports passkeys (the Web Authentication standard), the student's first scan creates a key pair in the phone's secure hardware. The private key never leaves the phone; the server stores only the public key. Every later scan requires the student to unlock the passkey with the phone's face recognition, fingerprint, PIN or pattern, and the server verifies the signature against the stored public key. Because the passkey is kept by the phone's operating system rather than in the site data of the browser, clearing the browser data does not break the binding. Passkeys saved to a password manager such as Google Password Manager or iCloud Keychain may synchronize to the student's other devices; the binding is therefore to the student's passkey rather than strictly to one handset, which is still considerably harder to share than the current identifier.
2. **Browser identifier as backup only.** The current random identifier is used only on phones and browsers that do not support passkeys. Students cannot choose the weaker method: a student bound by passkey can be moved to the backup method only after an instructor or the superadministrator clears the binding. The method in use is derived from whether the student has a registered passkey and is shown to instructors.
3. **Private-mode warning.** The scanner page checks whether the browser can keep the device identifier. When it cannot, as in private or incognito browsing, the page warns the student to scan from a normal browser tab before the scan is attempted.

The passkey part adds a `passkey_credential` table with the columns `credential_id` (primary key), `user_id` (foreign key to `user`), `public_key`, `sign_count`, `created_at` and `last_used_at`. The binding method is not stored as a

separate column, because it can be derived from the existence of a credential, which avoids redundant data. The existing `device_id` column of `user` is kept for the backup method. The rotating QR code, geofencing, enrollment check, lateness rule, exports and dashboards are not affected; the only change to the scanning process is the unlock step, which adds a few seconds to each scan. Passkeys require a secure (HTTPS) connection, which the deployment already provides. The private-mode warning requires no database change.

12.5 Student Search and Report Page

The planned version adds a read-only student search and report page to the superadministrator dashboard. The superadministrator selects filters from lists (year level, course or program, account status and subject), a sort order (by name, identification number or attendance rate) and a searching method (linear or binary search). The page uses the algorithms already implemented in `attendance/dsa.py` (merge sort, binary search, linear search, counting and ranking) and displays the number of comparisons performed, as the command-line interface does. Because departments are linked to subjects rather than to students in the database, students are selected by department through the subjects in which they are enrolled. The page requires no change to the database.

A free-form box in which the superadministrator could type Python code to query the database was considered and rejected. Code typed into a web page runs with the full permissions of the web application, so anyone who obtained the superadministrator's password could read the secret key, alter or delete the database, or change the files of the hosting account; restricting such code reliably is not possible. A long-running command would also occupy the single web worker of the hosting plan and delay scanning for every class in session. Such a box would also bypass the validation, soft-delete rule and audit trail that govern all changes made through the system. Direct database queries therefore remain available only from the server console, to the holder of the hosting account. The planned page offers only predefined, read-only operations and preserves these safeguards.

12.6 Geofence Default of 250 Meters

During the live demonstration, most students were refused as out of bounds although they were in the classroom. The likely causes are the inaccuracy of locations measured indoors and, when a session is started on a laptop, the position estimated from the Wi-Fi network or internet connection rather than from satellite positioning, which can

be off by hundreds of meters. The planned version raises the default radius from 40 to 250 meters, so that the geofence confirms that the student is near the classroom building rather than inside the room. Instructors may still set any radius from 10 to 1,000 meters for a particular session or turn geofencing off. The verification will also take into account the accuracy reported by both devices, up to a fixed limit, and the refusal message will state the measured distance and the allowed radius so that a problem can be identified on the spot; instructors will be advised to start sessions on a phone. The larger radius means that a code relayed to a student in a nearby building would be accepted. This trade-off is accepted because the primary goal of the system is an efficient attendance process; device binding continues to prevent one phone from checking in for several students. Limitations 6 and 9 and the description of the geofence in Section 4 will be revised accordingly. The radius is already stored for each session, so no database change is required.

12.7 QR Code Validity of 30 Seconds and Faster Roster Refresh

In the current version each QR code is valid for ten seconds. During the live demonstration, students on slow connections were frequently refused with an expired or invalid code, which made the attendance process slower rather than faster. The planned version makes each code valid for 30 seconds and refreshes the code on the instructor's screen every 30 seconds, with the countdown adjusted to match. The longer validity also suits online classes, in which geofencing is turned off and students scan the code from a shared screen. A code captured on a screenshot remains usable for up to 30 seconds; this is accepted for the same reason as the larger geofence. The live roster on the instructor's screen will refresh every 10 seconds instead of every 30, so that check-ins appear sooner. A forty-student roster is about ten kilobytes, so this adds about 3.6 megabytes of traffic per instructor per hour, which the hosting plan can absorb.

12.8 Pop-Up Notification for Unenrolled Scans

When a student who is not enrolled in the subject scans the code, the current version lists the student in the approval queue of the live session screen, where the instructor can add the student to the class and record the attendance. The planned version also shows a pop-up notification on the instructor's screen when a new unenrolled scan arrives, with an Approve button that performs the existing approval and a Refuse button that removes the student from the queue. A refused scan is marked so that it does not reappear at the next refresh, which adds one column to `audit_scan_log`; the column

describes only the scan in its own row, so the table remains in Third Normal Form. The notification appears within the roster refresh interval of 10 seconds.

12.9 Password Recovery

The current version has no way for a user who forgets the password to regain access except through the superadministrator. The planned version adds a "Forgot password?" link to the sign-in page. The user enters the e-mail address of the account and receives a link that opens a page for choosing a new password. The link can be used only once, expires after one hour and stops working when the password is changed, and the page always answers that a link was sent if the address is registered, so that it does not reveal which addresses have accounts. The e-mail is sent through a Gmail account with an application password stored in the server configuration, because the free hosting plan permits outgoing mail only through Gmail or web-based mail services. To make sure that the address is correct, new accounts will confirm their e-mail address through a link sent at registration.

As a last resort, for a user who entered a wrong e-mail address or no longer has access to it, the superadministrator can generate the same kind of reset link from the user's account page after verifying the user's identity in person, and give it to the user. The superadministrator never sees or sets the user's password. The feature uses the existing e-mail, password and account status columns and requires no database change.

12.10 Student Class Feed

The planned version adds a class feed to the student's screen during a session. It lists the classmates who have checked in, together with the student's own status and the number of students checked in, and refreshes every 60 seconds. To protect the personal data of students, the feed does not show which classmates were marked late or absent; those records remain visible only to the student concerned, the instructor and the superadministrator. The feed stops refreshing when the session closes or the page is not being viewed. A class of forty students refreshing every 60 seconds sends fewer than one request per second to the server, which the single web worker of the hosting plan can handle together with the scans. No database change is required.

12.11 Year Level Limited to 1 to 4

The sign-up form currently offers year levels 1 to 5, and the superadministrator's edit form accepts any positive number. Because the programs of the institution last four

years, the planned version limits the year level to 1 to 4 on the sign-up form, the superadministrator's edit form and the command-line interface. This is a validation rule only; the column and its data type are unchanged, and the data dictionary entry will state the allowed range.

REFERENCES

Codd, E. F. (1972). Further normalization of the data base relational model. In R. Rustin (Ed.), *Data base systems* (pp. 33–64). Prentice-Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Django Software Foundation. (2025). *Django* (Version 5.2) [Computer software]. https://www.djangoproject.com/

International Organization for Standardization & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011).

International Organization for Standardization & International Electrotechnical Commission. (2015). *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification* (ISO/IEC Standard No. 18004:2015).

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Official Gazette of the Republic of the Philippines.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.